

TEAM MANAGEMENT PLAYBOOK

Discord Server Setup Guide

Complete setup manual for marketing team leaders — server architecture, custom bot, daily workflows, and performance tracking in one comprehensive guide.

PREPARED FOR

Marketing Team Leaders

EDITION

v1.0 — July 2026

SCOPE

Server Setup + Custom Bot + Workflows



Table of Contents

Table of Contents	2
Chapter 1: Introduction & Overview	5
1.1 Purpose of This Guide	5
1.2 What You Will Build	5
1.3 Who Should Use This Guide	5
1.4 Expected Outcomes	6
Chapter 2: Server Architecture Blueprint	6
2.1 The Mixed Organizational Model	6
2.2 Category Map	6
2.3 Channel Naming Conventions	7
2.4 Voice Channels Strategy	8
Chapter 3: Roles & Permissions System	8
3.1 The Five-Tier Hierarchy	8
3.2 Permissions Matrix by Category	8
3.3 Setting Up Channel-Level Permission Overrides	9
3.4 Role Colors and Visual Identity	9
Chapter 4: Category-by-Category Setup	10
4.1 Information Hub	10
4.2 Attendance	11
4.3 Task Management	11
4.4 Marketing Departments	12
4.5 Projects	12
4.6 Reports	12
4.7 Performance	12
4.8 Meetings	13
4.9 Team Leader	13
Chapter 5: Custom Discord Bot Setup	13
5.1 Why a Custom Bot?	13

5.2 Prerequisites	14
5.3 Step 1 — Create the Bot Application	14
5.4 Step 2 — Invite the Bot to Your Server	14
5.5 Step 3 — Install Python Dependencies	15
5.6 Step 4 — Configure the Bot	15
5.7 Step 5 — Run the Bot	16
5.8 Keeping the Bot Running	16

Chapter 6: Bot Commands Reference **16**

6.1 Attendance Commands	17
6.2 Task Management Commands	17
6.3 Report Commands	18
6.4 Meeting Commands	18
6.5 Performance Commands	18
6.6 Knowledge Base Commands	19
6.7 Help Command	19

Chapter 7: Daily Workflow & Best Practices **19**

7.1 The Daily Cadence	19
7.2 Team Leader's Daily Checklist	20
7.3 Communication Etiquette	20
7.4 Onboarding New Members	21

Chapter 8: Performance Tracking & KPIs **21**

8.1 The Points System	21
8.2 Key Performance Indicators	22
8.3 Using the Leaderboard	22
8.4 Quarterly Review Template	23

Chapter 9: Meeting Management **23**

9.1 Meeting Types and Frequency	23
9.2 Scheduling with the Bot	23
9.3 Running a Daily Standup	24
9.4 Weekly Team Sync Agenda	24
9.5 Meeting Notes and Follow-up	24

Chapter 10: Knowledge Base & Documentation

24

10.1 Why a Knowledge Base Matters

24

10.2 The Six Category Tags

25

10.3 Contributing to the Knowledge Base

25

10.4 Searching the Knowledge Base

25

10.5 Maintenance and Pruning

26

Chapter 11: Troubleshooting & Maintenance

26

11.1 Common Issues and Solutions

26

11.2 Routine Maintenance Tasks

27

11.3 Data Backup

27

11.4 Bot Updates

27

Appendix A: Complete File Structure

28

Appendix B: Quick Start Checklist

28

Phase 1: Discord Server Setup (45 minutes)

28

Phase 2: Bot Setup (45 minutes)

29

Phase 3: Team Launch (30 minutes)

29

Phase 4: First Week Monitoring (ongoing)

30

Chapter 1: Introduction & Overview

1.1 Purpose of This Guide

This guide walks you through the complete process of setting up a Discord server tailored for marketing team leaders who need to monitor team members, assign daily tasks, track attendance, collect end-of-day reports, and evaluate performance. Rather than relying on scattered tools across multiple platforms, Discord provides a single centralized hub where your team can communicate, collaborate, and stay accountable — all in real time.

The guide is designed for teams of six to fifteen members, which is the sweet spot for Discord-based collaboration: large enough to need structure and clear channels, but small enough that a single Team Leader can actively monitor everyone. We will use a mixed organizational structure that combines functional departments (social media, content, SEO, paid ads, email) with project-based channels, giving you the flexibility to spin up new project channels without disrupting the department-based workflows.

1.2 What You Will Build

By the end of this guide, you will have a fully functional Discord server with a custom Python bot that automates the most repetitive parts of team leadership. The system includes:

- **Server architecture** — nine categories with twenty-plus channels, organized by purpose (information, attendance, tasks, departments, projects, reports, performance, meetings, leader-only).
- **Roles and permissions** — five-tier hierarchy (Team Leader, Senior Marketer, Specialist, Junior, Guest) with channel-level permission overrides.
- **Custom Discord bot** — a Python bot with slash commands for check-in/check-out, task management, daily reports, meeting scheduling, performance tracking, and knowledge base tagging.
- **Automated reminders** — morning check-in reminder, end-of-day report reminder, and auto-generated weekly summary every Friday.
- **Gamified performance system** — points awarded for punctual check-ins, task completion (with bonus for early completion), report submissions, meeting attendance, and peer kudos.
- **Daily workflow playbook** — a recommended cadence for standups, task assignment, progress check-ins, and end-of-day reports that keeps the team aligned without micromanaging.

1.3 Who Should Use This Guide

This guide assumes you are already a Discord user (you have a Discord account and know how to create a server), but it does not assume any programming experience. The bot setup section walks through every step in detail, from creating a bot application on the Discord Developer Portal to installing

Python dependencies and running the bot for the first time. If you can install software and edit a JSON file, you can deploy this bot.

The guide is specifically written for marketing teams, but the architecture and bot translate directly to any team-based knowledge work: development teams, customer support teams, design agencies, content studios, and operations teams can all use the same setup with minor renaming of channels and roles.

1.4 Expected Outcomes

Teams that adopt this system typically see three improvements within the first month. First, attendance and punctuality improve because the daily check-in becomes a visible, gamified ritual — members earn points for being on time and lose the bonus for being late. Second, task visibility improves dramatically because every task is posted publicly in the `#task-assignments` channel with the assignee tagged, the deadline visible, and the priority color-coded. Third, daily reports become a habit rather than a chore because the bot reminds members at 17:00 and the structured fields (completed, in-progress, blockers, tomorrow) take only two minutes to fill out.

As a Team Leader, you gain a single dashboard (the `#attendance-logs`, `#task-progress`, and `#daily-reports` channels) where you can scan the entire team's day in under five minutes. The weekly summary that posts automatically every Friday afternoon gives you a quantitative view of who delivered what, which feeds directly into your one-on-one reviews and sprint retrospectives.

Chapter 2: Server Architecture Blueprint

2.1 The Mixed Organizational Model

Pure functional structures (one channel per department) work well for stable teams, but marketing teams frequently form cross-functional project squads that pull members from multiple departments. Pure project-based structures, on the other hand, fragment department knowledge and make it hard to maintain shared standards. The mixed model used in this guide solves both problems by maintaining five permanent department channels for ongoing discussions and adding project channels on demand for time-bound campaigns.

When a new campaign launches — say, a Q4 holiday push that involves social media, paid ads, and email marketing — you create a `#project-holiday-q4` channel. Members from all three departments join that channel for the duration of the campaign. Once the campaign concludes, you archive the channel. Department channels remain stable and serve as the long-term home for discipline-specific discussions, templates, and questions.

2.2 Category Map

The table below lists every category and channel in the recommended server layout. Create them in this exact order — Discord sorts categories alphabetically by default, but you can drag them into any order once created. The order shown here places the most-frequently-used categories (information,

attendance, tasks) at the top and the least-frequently-used (meetings, leader-only) at the bottom.

#	Category	Channel	Type	Purpose
1	INFORMATION	#announcements	text	Team-wide announcements, scheduled meetings
	HUB	#rules	text	Server rules and expectations
		#team-info	text	Team roster, time zones, contacts
		#knowledge-base	text	Tagged playbooks, templates
2	ATTENDANCE	#check-in	text	Bot posts morning reminders here
		#attendance-logs	text	Auto-logged check-in/check-out
3	TASK	#task-assignments	text	New tasks posted by bot
	MANAGEMENT	#task-progress	text	Discuss ongoing task progress
		#completed-tasks	text	Auto-logged completed tasks
4	MARKETING	#social-media	text	Social media discussions
	DEPARTMENTS	#content-creation	text	Blog, video, design content
		#seo-analytics	text	SEO and analytics
		#paid-ads	text	Paid advertising campaigns
		#email-marketing	text	Email campaigns and automation
5	PROJECTS	#project-alpha	text	Specific project discussions
		#project-beta	text	Add more as needed
6	REPORTS	#daily-reports	text	Auto-posted daily reports
		#weekly-summary	text	Auto-posted weekly summaries
7	PERFORMANCE	#leaderboard	text	Weekly leaderboard posts
		#kudos	text	Peer recognition
8	MEETINGS	#meeting-notes	text	Shared meeting notes
		Meeting Room	voice	Daily standups, team meetings
		Break Room	voice	Casual voice chat
9	TEAM LEADER	#tl-private	text	TL-only private discussions
		#tl-decisions	text	TL-only decision log

2.3 Channel Naming Conventions

Consistent naming makes the server scannable. Use lowercase kebab-case for all text channels (#task-assignments, not #TaskAssignments or #task_assignments). Use Title Case for voice channels

since they represent named rooms (Meeting Room, Break Room). Prefix project channels with #project- so they group together in the sidebar and are easy to archive in bulk when a sprint ends.

Avoid emoji in channel names during the initial setup — they look playful but they make channel references harder to type and parse. Once the team is comfortable with the layout, you can add a single leading emoji to each category (e.g., 📁 for INFORMATION, 📅 for ATTENDANCE) to aid visual scanning.

2.4 Voice Channels Strategy

Two voice channels are enough for a team of fifteen. The Meeting Room is the formal space for daily standups, weekly team meetings, and project reviews. The Break Room is the casual space where members can hang out, co-work silently, or pair on a task. If your team grows beyond fifteen, add a third channel called Focus Room for silent deep-work sessions, and consider adding project-specific voice channels under the PROJECTS category.

Chapter 3: Roles & Permissions System

3.1 The Five-Tier Hierarchy

Discord permissions are role-based, and roles stack — a member with two roles gets the union of their permissions. To keep things simple, this guide uses five non-overlapping roles arranged in a strict hierarchy. The hierarchy matters because Discord only lets a role manage roles below it in the list, which prevents Junior Marketers from promoting themselves or demoting peers.

Role	Color	Who	Key Permissions
Team Leader	Blurple	You + deputies	Administrator (full access), manage roles, manage channels, use all bot commands
Senior Marketer	Green	Experienced members	Manage messages, add KB entries, mute members, assign tasks via bot
Marketing Specialist	Yellow	Standard members	Send messages, use slash commands, voice chat, read all channels
Junior Marketer	Grey	New hires (first 30 days)	Send messages in most channels, read-only in #knowledge-base
Guest	Fuchsia	Clients, observers	Read-only access to #announcements and #team-info only

3.2 Permissions Matrix by Category

The matrix below shows which roles can do what in each category. Cells marked with a checkmark indicate the default permission; permission overrides on individual channels can narrow these further. The Team Leader column is omitted because the Team Leader role has Administrator permission and

can do everything everywhere.

Category	Senior	Specialist	Junior	Guest
INFORMATION	Read + Post	Read + Post	Read	Read announcements only
ATTENDANCE	Read + Post	Read + Post	Read + Post	No access
TASK MANAGEMENT	Read + Post + Assign	Read + Post	Read + Post	No access
MARKETING DEPARTMENTS	Read + Post	Read + Post	Read + Post	No access
PROJECTS	Read + Post	Read + Post	Read + Post	Per project
REPORTS	Read own + Post	Read own + Post	Read own + Post	No access
PERFORMANCE	Read	Read	Read	No access
MEETINGS	Read + Post + Voice	Read + Post + Voice	Read + Post + Voice	No access
TEAM LEADER	No access	No access	No access	No access

3.3 Setting Up Channel-Level Permission Overrides

Category-level permissions cascade to all channels in that category, but you can override them on individual channels. The most important override is on the #tl-private and #tl-decisions channels in the TEAM LEADER category: these channels must be invisible to everyone except Team Leaders. To set this up:

1. Right-click the channel → Edit Channel.
2. Go to the Permissions tab.
3. Click the + button next to "Roles / Members" and add @everyone.
4. Set "View Channel" to the red X (deny) for @everyone.
5. Click the + button again and add the Team Leader role.
6. Set "View Channel" to the green check (allow) for Team Leader.
7. Repeat for #tl-decisions.

NOTE Permission overrides are additive for allows and subtractive for denials. If @everyone is denied View Channel but a member's other role grants View Channel category-wide, the channel-level deny wins. Always test by switching to a test account or asking a Junior Marketer to verify they cannot see the private channel.

3.4 Role Colors and Visual Identity

Role colors serve as visual cues in the member sidebar and message author tags. Use the Discord brand palette for consistency: Blurple (#5865F2) for Team Leaders, Green (#57F287) for Senior Marketers, Yellow (#FEE75C) for Specialists, Grey (#949BA4) for Juniors, and Fuchsia (#EB459E) for Guests. Set "Display role members separately from other online members" to ON for the Team Leader role so they appear at the top of the member list — this makes it easy for team members to find you when they need help.

Chapter 4: Category-by-Category Setup

This chapter walks through each category in detail, explaining the purpose of every channel, the recommended topic text (shown when members hover over the channel name), and any permission overrides you should apply. Create categories in the order presented here — Discord lets you reorder them by dragging later, but creating them in order saves time.

4.1 Information Hub

This category is the team's reference layer. Members visit it on day one to learn the rules and return to it whenever they need to find a documented procedure. All channels here are read-only for everyone except Team Leaders and Senior Marketers, which keeps the content authoritative and prevents well-meaning members from editing pinned templates.

#announcements

Purpose: broadcast important updates to the entire team — scheduled meetings, deadline reminders, organizational changes, wins worth celebrating. The bot also posts scheduled meeting summaries here automatically. Recommended topic: "Team-wide announcements. Read only. Use @here for time-sensitive items, @everyone only for true emergencies."

#rules

Purpose: codify the team's working agreement. Pin a single message containing the rules: expected response time during working hours, daily check-in deadline, daily report deadline, code of conduct, escalation path for blockers, and confidentiality expectations. Recommended topic: "Server rules and team working agreement. Read on day one, refer back when in doubt."

#team-info

Purpose: living roster with each member's name, role, time zone, primary contact method, and areas of expertise. Pin a table-formatted message and update it whenever the team changes. Recommended topic: "Who's who on the team. Time zones, contact methods, specialties."

#knowledge-base

Purpose: searchable repository of marketing playbooks, brand guidelines, campaign templates, tool tutorials, and case studies. Use the bot's /kb-add command to post entries with category tags (Playbook, Template, Brand Guide, Tool Tutorial, Case Study, FAQ). Members can search by typing

keywords into the channel search bar. Recommended topic: "Tagged KB entries. Use /kb-add to contribute. Search by tag: [Playbook], [Template], etc."

4.2 Attendance

This category hosts the daily check-in ritual. Members run /checkin when they start work and /checkout when they finish. The bot logs each event to #attendance-logs with the member's name, time, status (on time or late), and points earned. As Team Leader, you can run /attendance at any time to see who has and has not checked in today.

#check-in

Purpose: the channel where the bot posts the morning check-in reminder at 09:00 and where members can post informal "good morning" messages. The actual /checkin command can be run from any channel — the bot will respond in the channel where the command was issued, but the log goes to #attendance-logs regardless. Recommended topic: "Daily check-in channel. Bot posts 09:00 reminder. Run /checkin anywhere."

#attendance-logs

Purpose: bot-only channel that records every check-in and check-out event as an embed. Members can read this channel to see who is online, but only Team Leaders can post in it (the bot bypasses this restriction because it has Manage Messages permission). Recommended topic: "Auto-logged attendance. Read-only for members. Use /attendance for the summary view."

4.3 Task Management

This category is the operational core of the server. Every task the team works on is tracked here from creation through completion. The bot's task commands (/task-create, /task-assign, /task-complete, /task-list) integrate with these channels to keep a permanent, searchable record of all work.

#task-assignments

Purpose: bot posts every newly-created task here as a color-coded embed (color reflects priority: blue for medium, yellow for high, red for urgent). The assignee is tagged in the message so they get an instant notification. Recommended topic: "New tasks appear here. Bot tags assignees automatically. Discuss progress in #task-progress."

#task-progress

Purpose: free-form discussion channel where members post updates, ask for help, and share screenshots or links related to in-progress tasks. This is the day-to-day conversation channel for active work. Recommended topic: "Discuss active tasks. Share screenshots, ask questions, coordinate handoffs."

#completed-tasks

Purpose: bot posts every completed task here with the assignee's avatar, completion time, and points earned. Use this channel for the Friday wrap-up — scroll through the week's completions to celebrate wins and identify patterns. Recommended topic: "Auto-logged completed tasks. Use for weekly review and celebrations."

4.4 Marketing Departments

These five channels are the permanent homes for discipline-specific conversations. Members tend to live primarily in their department channel and visit project channels only when their work crosses over. Each channel should have at least one pinned message containing the department's working norms, key tools, and current focus areas.

If your team has different departments (e.g., PR, Events, Partnership), rename the channels accordingly — the structure scales. Avoid creating more than seven department channels; beyond that, the sidebar becomes cluttered and members start missing messages. If a department is small (one or two members), fold it into a related department rather than giving it its own channel.

4.5 Projects

Project channels are temporary — they exist for the duration of a campaign and are then archived. Naming convention: #project- (e.g., #project-summer-launch, #project-black-friday). When a project ends, right-click the channel → Edit Channel → Archive Channel. Archived channels disappear from the sidebar but remain searchable, which preserves institutional knowledge without cluttering the active view.

Resist the temptation to give every cross-team conversation its own project channel. A good rule of thumb: only create a project channel when the work will span at least one week and involve at least three members from two or more departments. For shorter or smaller collaborations, use a thread inside the relevant department channel.

4.6 Reports

These two channels capture the team's daily and weekly cadence. The #daily-reports channel is bot-driven: every time a member runs /report-submit, the bot formats their input into a structured embed and posts it here. The #weekly-summary channel receives the auto-generated Friday afternoon summary, which lists the week's top performers, total tasks completed, and tasks still open.

Make #daily-reports the first channel you check each morning and the last you check each evening. It gives you a complete picture of what the team delivered yesterday and what they plan to deliver tomorrow, in their own words. Reading these reports carefully is the single highest-leverage activity you can do as a Team Leader.

4.7 Performance

The #leaderboard and #kudos channels are the gamification layer. The bot posts the weekly leaderboard to #weekly-summary every Friday, but you can also pin a manually-refreshed leaderboard

message in #leaderboard. The #kudos channel is for peer-to-peer recognition: any member can run /kudos @member "reason" to publicly thank a teammate, and the bot awards the recipient two bonus points.

Gamification works best when it is transparent and consistent. Never adjust points manually — if a member feels the system is unfair, address the underlying rule rather than patching individual scores. Review the points configuration in config.json quarterly and adjust based on what behaviors you want to encourage.

4.8 Meetings

The Meetings category has one text channel (#meeting-notes) for shared notes and one or two voice channels for the actual meetings. During a meeting, one person (typically the Team Leader or a rotating note-taker) types key decisions, action items, and owners into #meeting-notes in real time. After the meeting, the note-taker formats the notes and pins them with a clear title (e.g., "Weekly Standup — 2026-07-03").

Use the bot's /meeting-schedule command to announce meetings in advance. The command posts a formatted embed to #announcements with the meeting title, type, time, duration, and agenda. Members who attend can run /meeting-attend to mark their presence and earn four points — this creates a soft incentive to show up on time.

4.9 Team Leader

This private category is your space as Team Leader. Use #tl-private for sensitive discussions with deputies or HR partners, and #tl-decisions as a running log of significant decisions (role changes, performance concerns, strategy shifts). The decision log becomes invaluable during performance reviews and quarterly planning — it is hard to remember six months later why a particular call was made unless you wrote it down at the time.

These two channels must be invisible to everyone except Team Leaders. Configure the permission overrides as described in section 3.3, and verify the configuration by asking a non-leader member to confirm they cannot see the channels in their sidebar.

Chapter 5: Custom Discord Bot Setup

5.1 Why a Custom Bot?

Off-the-shelf Discord bots like MEE6 and Carl-bot cover moderation and reaction roles, but they do not handle the team-management workflow you need: attendance tracking, task lifecycle, structured daily reports, performance points, and weekly summaries. A custom bot gives you complete control over the commands, the data format, and the points logic — and because the bot is yours, you can extend it with new commands as your workflow evolves.

The bot included with this guide is written in Python using discord.py 2.3+, the most widely-used Discord library. It uses slash commands (the modern command interface that Discord introduced in 2021), persists data to local JSON files (no database setup required), and includes a scheduled reminder system using APScheduler. The full source code is in bot.py — about 700 lines, well-commented, and easy to extend.

5.2 Prerequisites

Before you begin, make sure you have the following:

- **Python 3.10 or newer** — verify with `python --version`. Install from python.org if needed.
- **pip** — Python's package installer, comes bundled with Python.
- **A Discord account** with permission to manage the target server.
- **The bot source files** — bot.py, config.json, requirements.txt — delivered alongside this guide.
- **A text editor** — VS Code, Sublime, or even Notepad will work. You only need to edit config.json.

5.3 Step 1 — Create the Bot Application

Open the Discord Developer Portal at discord.com/developers/applications and sign in with your Discord account. Click the "New Application" button in the top-right corner, name your application (e.g., "Marketing Team Bot"), and click Create. The application is the container for your bot; one application maps to one bot.

Once the application is created, navigate to the Bot tab in the left sidebar and click "Add Bot". Confirm the prompt. You now have a bot user attached to your application. Scroll down to the Privileged Gateway Intents section and toggle ON both the Server Members Intent and the Message Content Intent — the bot needs these to read member information and (in some legacy commands) message content.

NOTE Treat your bot token like a password. Anyone with the token can run any command as your bot. Never commit it to a public Git repository — the included .gitignore file already excludes config.json, but double-check before pushing.

5.4 Step 2 — Invite the Bot to Your Server

Stay in the Developer Portal and navigate to OAuth2 → URL Generator. Under Scopes, check the "bot" checkbox and the "applications.commands" checkbox. The first authorizes the bot to join your server; the second authorizes it to register slash commands. Under Bot Permissions, check the following:

- Manage Roles
- Read Messages / View Channels
- Send Messages
- Embed Links

- Read Message History
- Mention Everyone
- Add Reactions
- Use Slash Commands

Copy the generated URL at the bottom of the page and open it in a new browser tab. Discord will ask which server to add the bot to — select your team server and click Authorize. Solve the CAPTCHA and the bot will appear in your server's member list, offline for now (it will come online once you run the Python script).

5.5 Step 3 — Install Python Dependencies

Open a terminal or command prompt in the folder where you placed the bot files. Create a virtual environment to keep dependencies isolated, then install the required packages:

```
bash # Create virtual environment python -m venv venv # Activate it # On macOS/Linux:
source venv/bin/activate # On Windows: venv\Scripts\activate # Install dependencies
pip install -r requirements.txt
```

5.6 Step 4 — Configure the Bot

Open config.json in your text editor. The file has five sections: token, channels, roles, schedule, and points. Fill them in as follows:

Token

Replace PUT_YOUR_BOT_TOKEN_HERE with the token you copied from the Developer Portal. Alternatively, set an environment variable named DISCORD_TOKEN and leave the token field in config.json as-is — the bot reads the env variable first.

Guild ID

Set guild_id to your server's ID. To find it, enable Developer Mode in Discord (Settings → Advanced → Developer Mode), then right-click your server name → Copy ID. Setting guild_id makes slash commands sync instantly to your server; without it, commands can take up to an hour to appear globally.

Channel IDs

Each channel listed in the channels section needs the actual channel ID from your server. Right-click each channel → Copy ID. If you did not create a particular channel yet, leave its value as 0 — the bot will skip posting to that channel without crashing.

Role IDs

Same process for roles. Right-click each role in the member list → Copy ID. The team_leader role ID is the most important — without it, the bot cannot tell who is allowed to use Team-Leader-only commands.

Schedule

Adjust the times in the schedule section to match your team's working hours. Times are in 24-hour format and use the timezone specified in the timezone field (default: Africa/Algiers). The four key times are: `morning_reminder` (when the bot reminds members to check in), `check_in_deadline` (the cutoff for on-time vs. late), `report_reminder` (when the bot reminds members to submit their daily report), and `report_deadline` (the cutoff for the day).

Points

The points section controls how many points each action awards. Defaults are balanced for a 40-hour work week: on-time check-in ($2 \text{ pts} \times 5 \text{ days} = 10$), daily report ($3 \text{ pts} \times 5 = 15$), one task per day ($5 \text{ pts} \times 5 = 25$), and meeting attendance ($4 \text{ pts} \times 1 = 4$) gives a typical active member about 54 points per week. Adjust the values if you want to emphasize different behaviors.

5.7 Step 5 — Run the Bot

With `config.json` filled in and dependencies installed, start the bot:

```
bash python bot.py
```

On a successful startup, you should see log output like this:

```
log 2026-07-03 09:00:00 | INFO | team-bot | Logged in as Marketing Team Bot#1234
2026-07-03 09:00:00 | INFO | team-bot | Connected to 1 guild(s) 2026-07-03 09:00:00 |
INFO | team-bot | Synced 16 slash commands to guild 123456789 2026-07-03 09:00:00 |
INFO | team-bot | APScheduler started for fine-grained scheduling
```

The bot is now online in your server. Type `/` in any channel to see the list of available commands. Try `/checkin` to verify attendance logging works, then `/help` to see the full command reference. The bot will continue running until you stop it with `Ctrl+C`.

5.8 Keeping the Bot Running

For day-to-day use, the bot should run continuously on a server rather than your laptop. The simplest option is a small cloud VM (DigitalOcean droplet, AWS EC2 t2.micro, or similar) running Linux. Copy the bot files to the VM, install Python and dependencies, and run the bot inside a terminal multiplexer like `tmux` or `screen` so it survives your SSH session disconnecting. For production deployments, consider using `systemd` to auto-restart the bot on crash or VM reboot — a sample `systemd` unit file is included in the README.

Chapter 6: Bot Commands Reference

The bot exposes sixteen slash commands organized into six functional groups. Every command is invoked by typing `/` followed by the command name in any channel where the bot can read and send messages. Most commands respond ephemerally (only the caller sees the response) to keep the channel clean; the bot simultaneously posts the permanent record (attendance log, task assignment,

etc.) to the appropriate log channel.

6.1 Attendance Commands

Command	Who	Parameters	Description
/checkin	All	—	Register daily check-in. Awards 2 pts on time, 1 pt late. Logs to #attendance-logs.
/checkout	All	—	Register check-out. Records work duration. Logs to #attendance-logs.
/attendance	TL	member (optional)	View today's attendance summary. Filter by member if provided.

Example: /checkin

Run /checkin in any channel. The bot responds privately:

```
bot response Check-In Successful You checked in ON TIME at 09:02:17. Points earned: +2
```

Simultaneously, in #attendance-logs, the bot posts:

```
#attendance-logs Check-In: John Doe Time: 09:02:17 Status: ON TIME Points: +2
```

6.2 Task Management Commands

Command	Who	Parameters	Description
/task-create	TL	title, description, deadline, priority?, assignee?	Create a new task. Posts to #task-assignments.
/task-assign	TL	task_id, member	Assign an existing task to a member.
/task-complete	Assignee / TL	task_id, notes?	Mark task complete. Awards 5 pts (+2 if early). Posts to #completed-tasks.
/task-list	All	status?, member?	List tasks; filter by status (open/completed/all) or assignee.

Example: /task-create

Suppose you want to assign John a high-priority task to draft the Q3 campaign brief by July 5 at 17:00. Run /task-create with these parameters:

```
command input title: Draft Q3 Campaign Brief description: Outline target audience, key messaging, channels, and budget for the Q3 launch campaign. Aim for 3-5 pages. deadline: 2026-07-05 17:00 priority: High assignee: @john
```

The bot creates the task, assigns ID #1, and posts to #task-assignments:

```
#task-assignments Task #1: Draft Q3 Campaign Brief Outline target audience, key messaging, channels, and budget for the Q3 launch campaign. Priority: HIGH Assignee:
```

@john Deadline: 2026-07-05 17:00 Created by: Team Leader

6.3 Report Commands

Command	Who	Parameters	Description
/report-submit	All	completed, in_progress, blockers?, tomorrow?	Submit daily report. Awards 3 pts. Posts to #daily-reports.
/report-view	TL	member?	View today's reports. Filter by member if provided.

Example: /report-submit

A typical daily report submission looks like:

```
command input completed: Drafted Q3 campaign brief; reviewed John's social media calendar; fixed three bugs in the email automation flow. in_progress: Finalizing the Q3 budget breakdown; waiting on design team for hero image assets. blockers: Need access to the Google Ads manager account – currently pending. tomorrow: Finish budget breakdown; start drafting the paid ads plan.
```

6.4 Meeting Commands

Command	Who	Parameters	Description
/meeting-schedule	TL	title, datetime_str, duration_minutes?, meeting_type?, agenda?	Schedule a meeting. Posts to #announcements.
/meeting-attend	All	meeting_id	Mark attendance. Awards 4 pts.

Example: /meeting-schedule

To schedule the weekly team meeting:

```
command input title: Weekly Team Sync datetime_str: 2026-07-07 14:00 duration_minutes: 45 meeting_type: Weekly Team Meeting agenda: Q3 campaign progress; budget review; task assignments for week of July 7
```

6.5 Performance Commands

Command	Who	Parameters	Description
/leaderboard	All	period?	Show top 10 performers. Period: weekly, monthly, or total.
/performance	All (TL for others)	member?	View detailed stats: points, attendance, tasks, reports.
/kudos	All	member, reason	Thank a teammate. Awards them 2 pts. Posts to #kudos.

6.6 Knowledge Base Commands

Command	Who	Parameters	Description
/kb-add	TL / Senior	title, url, category, summary?	Add a tagged KB entry. Posts to #knowledge-base.

Categories are: Playbook, Template, Brand Guide, Tool Tutorial, Case Study, FAQ. The bot tags each entry with the category name in the title (e.g., [Playbook] How to Run a Product Launch) so members can filter the channel by typing the tag in the search bar.

6.7 Help Command

Run /help at any time to see a categorized list of all available commands with one-line descriptions. The help response is ephemeral, so it does not clutter the channel.

Chapter 7: Daily Workflow & Best Practices

7.1 The Daily Cadence

A successful Discord-based team runs on a predictable daily cadence. The Team Leader's job is to set the rhythm, reinforce it for the first two weeks, and then step back and let the team self-regulate. The recommended cadence for a marketing team working 09:00 to 18:00 in a single time zone is shown below. Adjust the times to fit your team's hours.

Time	Activity	Owner	Tool / Channel
09:00	Bot sends morning check-in reminder	Bot (automatic)	#check-in
09:00–10:00	Members check in	All members	/checkin command
10:00	Check-in deadline (late thereafter)	Bot (automatic)	Points deduction
10:15	Team Leader reviews attendance	Team Leader	/attendance
10:15	Team Leader assigns daily tasks	Team Leader	/task-create
10:30	Brief morning standup (optional)	Team Leader + volunteers	Meeting Room
Throughout day	Members work on tasks	All members	#task-progress
14:00	Mid-day progress check (optional)	Team Leader	Scan #task-progress
17:00	Bot sends report reminder	Bot (automatic)	#daily-reports
17:00–18:00	Members submit daily reports	All members	/report-submit
18:00	Report deadline	Bot (automatic)	—

Time	Activity	Owner	Tool / Channel
18:00	Members check out	All members	/checkout
Friday 16:00	Bot posts weekly summary	Bot (automatic)	#weekly-summary
Friday 16:30	Weekly team retro	Team Leader + all	Meeting Room

7.2 Team Leader's Daily Checklist

As Team Leader, your day has four high-leverage moments. Spending five to ten minutes at each of these moments is enough to keep the team aligned without micromanaging.

Morning (10:00–10:30)

- Run /attendance to see who has checked in. Note anyone late or absent.
- Scan #task-progress for overnight questions or blockers that need your input.
- Run /task-create for any new tasks that came in overnight. Assign immediately.
- Post a brief morning message in #announcements summarizing the day's priorities.

Mid-day (14:00–14:30)

- Scan #task-progress for stalled conversations or questions that went unanswered.
- Check #task-assignments for any tasks that have not been picked up.
- Drop into the Meeting Room voice channel if anyone is working there and needs a quick sync.

End-of-day (17:30–18:00)

- Run /report-view to read each member's daily report.
- Reply directly (via DM or in #task-progress) to any report that flags a blocker.
- Note any tasks marked complete in #completed-tasks and acknowledge them publicly.
- Run /attendance one final time to confirm everyone checked out.

Friday (16:00–17:00)

- Read the auto-posted weekly summary in #weekly-summary.
- Run /leaderboard weekly to see the points ranking. Mention the top performer in #announcements.
- Host a 30-minute weekly retro in the Meeting Room. Use #meeting-notes for live notes.
- Plan the following week's major tasks so you can assign them Monday morning.

7.3 Communication Etiquette

Discord can become noisy quickly if the team does not agree on communication norms. The following etiquette should be posted in #rules on day one and reinforced for the first two weeks. After that, it becomes self-enforcing.

- **Use threads for any conversation longer than three messages.** Threads keep the main channel scannable and let people opt out of discussions that do not concern them.
- **@mention sparingly.** Reserve @here for time-sensitive items that need attention within the hour. Reserve @everyone for true emergencies. For individuals, use @name only when you need their direct input — not just to acknowledge that you saw their message.
- **Post updates in the right channel.** Task progress goes in #task-progress, not in #social-media. Department-specific questions go in the department channel. General announcements go in #announcements.
- **Respect working hours.** Do not @mention anyone outside their stated working hours unless it is a true emergency. Discord's Do Not Disturb mode is a signal to wait until morning.
- **Be explicit about blockers.** If you are blocked, say so in #task-progress and tag the Team Leader. Suffering in silence helps no one.

7.4 Onboarding New Members

When a new member joins, follow this five-step onboarding checklist. The first day sets the tone for their entire tenure on the team, so invest thirty minutes up front to save hours of confusion later.

1. Assign the Junior Marketer role. Walk them through the server structure (categories, channels, voice rooms).
2. Have them read #rules and #team-info. Answer any questions.
3. Show them the /help command and walk through /checkin, /report-submit, and /task-list.
4. Assign them one small task using /task-create so they can practice the workflow end-to-end.
5. Pair them with a Senior Marketer as a buddy for their first two weeks. The buddy answers day-to-day questions and reviews their first three daily reports.

Chapter 8: Performance Tracking & KPIs

8.1 The Points System

The bot's points system is the quantitative backbone of performance tracking. Every positive behavior earns points; the weekly and monthly leaderboards make those points visible to the entire team. The points are not a substitute for substantive performance reviews — they are a leading indicator that helps you spot patterns early and start conversations before small issues become big ones.

Action	Points	Notes
On-time check-in (before deadline)	+2	Daily, up to 10 pts/week
Late check-in (after deadline)	+1	Half credit, no penalty
Task completed	+5	Per task, regardless of size
Task completed early (before deadline)	+7	Bonus for proactive delivery
Daily report submitted	+3	Daily, up to 15 pts/week
Meeting attended	+4	Per meeting
Kudos received from peer	+2	Capped at 10 pts/week from kudos

A typical active member who checks in on time every day, submits five reports, completes five tasks (one early), attends one meeting, and receives three kudos earns: $10 + 15 + 25 + 2 + 4 + 6 = 62$ points per week. Use this as a baseline when reviewing individual performance — anyone significantly below 40 points per week is either disengaged, blocked, or unaware of available tasks, and warrants a one-on-one conversation.

8.2 Key Performance Indicators

Beyond raw points, the bot tracks four KPIs that together paint a complete picture of each member's contribution. Review these in your weekly one-on-ones and quarterly performance reviews.

KPI	What It Measures	Healthy Range	Action if Below
Attendance rate	% of working days checked in	$\geq 95\%$	Discuss time management; check for burnout
Punctuality rate	% of check-ins on time	$\geq 85\%$	Investigate scheduling conflicts; review workload
Task completion rate	% of assigned tasks completed	$\geq 90\%$	Review task scoping; check for blockers
Report consistency	% of working days with report submitted	$\geq 95\%$	Reinforce habit; pair with buddy

8.3 Using the Leaderboard

The leaderboard is a tool for recognition, not punishment. Use it to celebrate wins and spot quiet contributors — the members who consistently deliver without making noise. Avoid using it to shame low performers; that creates a hostile culture and incentivizes gaming the system (e.g., checking in then disappearing).

In your Friday wrap-up message, mention the top three performers by name and call out the specific behavior that earned their points ("John led the week with 78 points, driven by completing three tasks early and submitting all five daily reports on time"). This is more meaningful than just announcing the

ranking and reinforces the behaviors you want to see.

8.4 Quarterly Review Template

Every quarter, run a structured review with each member using the bot's data as a starting point. The review has four parts:

- **Quantitative review** — pull /performance for the past 13 weeks. Look at trends: is attendance stable, declining, or improving? Is task completion rate steady? Are reports consistent?
- **Qualitative review** — review the member's daily reports from the quarter. What did they learn? What did they struggle with? What were their biggest wins?
- **Goal-setting** — set three goals for the next quarter: one skill to develop, one process to improve, one outcome to deliver. Document these in #tl-decisions.
- **Feedback** — ask the member for feedback on your leadership and on the team's workflow. Listen without defending. Adjust based on what you hear.

Chapter 9: Meeting Management

9.1 Meeting Types and Frequency

Meetings are expensive — a 30-minute meeting with eight people consumes four person-hours of work. Run only the meetings that have a clear purpose, and keep them tight. The recommended meeting cadence for a marketing team of fifteen is:

Meeting	Frequency	Duration	Attendees	Purpose
Daily standup	Mon–Thu	10 min	All	Quick round-robin: yesterday, today, blockers
Weekly team sync	Friday	45 min	All	Week review, next week planning, kudos
Project review	Per project	30 min	Project squad	Status, risks, decisions
1-on-1	Bi-weekly	30 min	TL + member	Personal check-in, growth, feedback
Quarterly review	Quarterly	60 min	TL + member	Performance, goals, development

9.2 Scheduling with the Bot

Use /meeting-schedule to announce any meeting at least 24 hours in advance. The bot posts the meeting details to #announcements with the title, type, time, duration, and agenda. Members who attend run /meeting-attend to record their presence and earn four points. This creates a soft attendance incentive and gives you a quantifiable attendance record for each meeting.

9.3 Running a Daily Standup

The daily standup is the team's most frequent meeting, so it must be efficient. Hold it in the Meeting Room voice channel at the same time every day (10:30 is a good slot — after check-in, before deep work). The format is simple: each member answers three questions in under two minutes.

- **Yesterday:** What did you complete yesterday?
- **Today:** What are you working on today?
- **Blockers:** Is anything blocking your progress?

As Team Leader, your job is to keep the standup moving. If a conversation needs more than two minutes, ask the relevant members to take it to a thread in #task-progress after the standup. Do not let the standup become a status report — status belongs in daily reports. The standup is for alignment and unblocking.

9.4 Weekly Team Sync Agenda

The Friday weekly sync is the team's most important meeting. Use it to close out the week, celebrate wins, and set up the next week. A 45-minute agenda that works well:

- **0:00–0:05 — Icebreaker:** Quick round-robin with a fun question ("best thing that happened this week").
- **0:05–0:15 — Week in review:** Walk through #completed-tasks. Call out notable wins.
- **0:15–0:25 — Metrics review:** Share any marketing metrics (traffic, conversions, engagement) from the week.
- **0:25–0:35 — Next week's priorities:** Discuss the top three priorities for next week. Assign owners.
- **0:35–0:40 — Blockers and asks:** Anyone blocked? Anyone need help?
- **0:40–0:45 — Kudos and leaderboard:** Announce the weekly leaderboard top three. Give kudos.

9.5 Meeting Notes and Follow-up

Designate a note-taker for every meeting (rotate the role weekly). The note-taker types key decisions and action items into #meeting-notes in real time, then formats and pins the notes at the end of the meeting with a clear title. After the meeting, the Team Leader creates tasks from the action items using /task-create, so every action item has an owner and a deadline tracked in the system.

Chapter 10: Knowledge Base & Documentation

10.1 Why a Knowledge Base Matters

Marketing teams generate enormous amounts of tacit knowledge: which ad copy converts best, which email subject lines get the highest open rates, which design templates match the brand voice, which analytics dashboards matter. Without a structured knowledge base, this knowledge lives in individual heads and Slack DMs, and it walks out the door when team members leave. A well-maintained #knowledge-base channel turns tacit knowledge into explicit, searchable institutional memory.

10.2 The Six Category Tags

The bot's /kb-add command requires a category tag for every entry. The six categories are deliberately broad — narrower tags lead to fragmentation and inconsistent tagging. If an entry does not fit one of the six, it probably belongs in a different channel.

Tag	What Goes Here	Example
[Playbook]	Step-by-step procedures for recurring marketing activities	How to launch a paid social campaign
[Template]	Reusable templates for documents, emails, ad copy	Q3 campaign brief template
[Brand Guide]	Brand standards: voice, tone, colors, fonts, logo usage	Brand voice do's and don'ts
[Tool Tutorial]	How-to guides for marketing tools (Google Analytics, Mailchimp, etc.)	Setting up conversion tracking in GA4
[Case Study]	Post-mortems of completed campaigns with metrics	Spring 2026 email campaign — what worked
[FAQ]	Answers to commonly asked questions from clients or team	How to request design assets

10.3 Contributing to the Knowledge Base

Team Leaders and Senior Marketers can add entries using /kb-add. The command takes a title, a URL (link to a Google Doc, Notion page, Loom video, or external resource), the category, and an optional summary. The bot formats the entry as an embed and posts it to #knowledge-base.

Encourage all team members to suggest KB entries by posting in #task-progress. When a member has a question that takes more than a few sentences to answer, the answer is probably KB-worthy. The Senior Marketer or Team Leader can then formalize it into a KB entry. This balances quality control (only Seniors and TLs can post) with broad contribution (anyone can suggest).

10.4 Searching the Knowledge Base

Discord's search bar at the top right of the channel supports keyword search, user search, date filters, and channel filters. To search the knowledge base, click the search bar, type "in:#knowledge-base ", and press Enter. To filter by tag, include the tag in your search (e.g., "in:#knowledge-base [Playbook] campaign").

Pin the five to ten most-frequently-referenced entries using Discord's pin feature. Pinned messages appear in a dedicated panel accessible by clicking the pin icon at the top of the channel, which makes them instantly findable even for new members who do not know what to search for.

10.5 Maintenance and Pruning

Schedule a quarterly KB review (add it to your calendar as a recurring event). During the review, scan all entries from the past quarter. Archive outdated entries (delete the message or move to a #kb-archive channel), update entries whose content has changed, and consolidate duplicate entries. A bloated knowledge base is almost as bad as no knowledge base, because members cannot find what they need.

Chapter 11: Troubleshooting & Maintenance

11.1 Common Issues and Solutions

Bot not responding to slash commands

If the bot is online (green dot in the member list) but slash commands do not appear or do not respond, check the following in order:

- Verify the bot was invited with the applications.commands scope. If you only checked "bot" in the OAuth2 URL generator, slash commands will not register. Re-invite the bot with both scopes.
- Verify guild_id in config.json is set to your server's ID. Without it, commands register globally and can take up to an hour to appear.
- Restart the bot. Slash command syncing happens on startup; if you changed config.json, the bot needs a restart.
- Check the bot log (bot.log) for sync errors. Common causes: missing permissions, malformed command definitions, or duplicate command names.

Permission errors

If the bot posts error messages about missing permissions, the most common cause is role hierarchy: Discord only lets a role manage roles below it. Make sure the bot's role is placed at the top of the role list (or at least above all the roles it needs to interact with). Also verify the bot has the permissions you granted during invitation — if a server admin changed permissions after the bot was invited, you may need to re-invite with the correct permissions.

Members cannot see a channel

If a member reports they cannot see a channel they should be able to, check the channel's permission overrides (right-click channel → Edit Channel → Permissions). Look for a deny on "View Channel" for either @everyone or the member's role. Common mistake: denying View Channel for @everyone on a public channel accidentally hides it from everyone.

Attendance not registering

If `/checkin` runs but no embed appears in `#attendance-logs`, verify the `attendance_logs` channel ID in `config.json` is correct (right-click channel → Copy ID). Also verify the bot has "Send Messages" and "Embed Links" permissions in that channel. If the bot can post in other channels but not this one, the issue is channel-specific permissions.

Bot crashes on startup

If the bot crashes immediately on startup, the most common cause is a missing or invalid `config.json`. Verify that the token field is set to your actual bot token (not the placeholder), that `guild_id` is a valid integer, and that all channel and role IDs are integers (not strings). Check `bot.log` for the specific error message.

11.2 Routine Maintenance Tasks

Frequency	Task	How
Daily	Read daily reports	Scan <code>#daily-reports</code>
Daily	Acknowledge completed tasks	React to <code>#completed-tasks</code> posts
Weekly	Run <code>/leaderboard</code> weekly	Announce top performers in <code>#announcements</code>
Weekly	Archive finished project channels	Right-click → Archive Channel
Monthly	Review role assignments	Promote Juniors ready for Specialist
Monthly	Audit channel permissions	Verify private channels still private
Monthly	Backup data files	Copy data/ folder to cloud storage
Quarterly	Review points configuration	Adjust values in <code>config.json</code> if needed
Quarterly	KB pruning	Archive outdated KB entries
Quarterly	Conduct 1-on-1 reviews	Use <code>/performance</code> data as starting point

11.3 Data Backup

The bot stores all data in JSON files in the `data/` directory: `attendance.json`, `tasks.json`, `reports.json`, `points.json`, and `meetings.json`. These files are the team's institutional memory — losing them means losing months of performance data and historical reports. Back them up at least monthly by copying the entire `data/` folder to cloud storage (Google Drive, Dropbox, S3). For teams that rely heavily on the data, set up an automated daily rsync or cron job to copy the files to a remote location.

11.4 Bot Updates

When you update `bot.py` (either to apply a fix from this guide's author or to add your own features), follow this safe-update procedure: stop the bot, back up the `data/` folder, replace `bot.py` with the new

version, restart the bot, and immediately run `/checkin` and `/task-list` to verify the bot is functioning. If anything looks wrong, stop the bot, restore the old `bot.py`, and restart. Never edit `bot.py` while the bot is running — the file may be partially written when the bot tries to reload it.

Appendix A: Complete File Structure

The bot is delivered as a single folder containing five files plus a data directory that is auto-created on first run. This appendix describes each file's purpose and what you should and should not edit.

File	Editable?	Purpose
<code>bot.py</code>	Yes (advanced)	Main bot file. Contains all commands, scheduled tasks, and the bot's startup logic. Edit only if you want to add or modify commands.
<code>config.json</code>	Yes (required)	Configuration: bot token, channel IDs, role IDs, schedule, points. Must be edited before first run.
<code>requirements.txt</code>	No	List of Python dependencies. Used by <code>pip install -r requirements.txt</code> . Do not edit unless you add new dependencies.
<code>README.md</code>	No	Quick-start documentation. Reference if you need to remember how to set up the bot.
<code>setup_helper.py</code>	No	Optional helper script. Prints all channel and role IDs from your server so you can copy them into <code>config.json</code> .
<code>.gitignore</code>	No	Tells Git to ignore <code>data/</code> , <code>venv/</code> , and <code>config.json</code> (to protect your token). Do not edit.
<code>bot.log</code>	No (auto)	Auto-generated log file. Useful for debugging. Rotate or delete periodically to save disk space.
<code>data/attendance.json</code>	No (auto)	Auto-generated. Stores all attendance records.
<code>data/tasks.json</code>	No (auto)	Auto-generated. Stores all tasks.
<code>data/reports.json</code>	No (auto)	Auto-generated. Stores all daily reports.
<code>data/points.json</code>	No (auto)	Auto-generated. Stores all points data.
<code>data/meetings.json</code>	No (auto)	Auto-generated. Stores all meetings.

Appendix B: Quick Start Checklist

Use this one-page checklist to track your progress through the full setup. Print it out and check off each step as you complete it. Estimated total time: 2–3 hours for a team of fifteen.

Phase 1: Discord Server Setup (45 minutes)

- ☐ Create the Discord server (or use existing one)

- ☐ Create all 9 categories in order (Information, Attendance, Tasks, Departments, Projects, Reports, Performance, Meetings, TL)
- ☐ Create all channels under each category (refer to Chapter 2 table)
- ☐ Set channel topics for each channel
- ☐ Create the 5 roles (Team Leader, Senior, Specialist, Junior, Guest) in order
- ☐ Assign role colors (Blurple, Green, Yellow, Grey, Fuchsia)
- ☐ Configure permission overrides on #tl-private and #tl-decisions
- ☐ Invite team members and assign roles

Phase 2: Bot Setup (45 minutes)

- ☐ Install Python 3.10+ on your machine or server
- ☐ Download bot files (bot.py, config.json, requirements.txt) to a folder
- ☐ Create a Discord application at discord.com/developers/applications
- ☐ Add a bot to the application; copy the bot token
- ☐ Enable Server Members Intent and Message Content Intent
- ☐ Generate OAuth2 URL with bot + applications.commands scopes and required permissions
- ☐ Open the URL and authorize the bot to join your server
- ☐ Open terminal in the bot folder; create virtual environment (python -m venv venv)
- ☐ Activate virtual environment and run pip install -r requirements.txt
- ☐ Enable Developer Mode in Discord; copy server ID, channel IDs, role IDs
- ☐ Edit config.json: set token, guild_id, all channel IDs, all role IDs
- ☐ Adjust schedule times and timezone to match your team
- ☐ Run python bot.py and verify startup logs
- ☐ Test /checkin and /help commands in Discord

Phase 3: Team Launch (30 minutes)

- ☐ Post welcome message in #announcements explaining the new system
- ☐ Pin the rules in #rules (use template from Chapter 4)
- ☐ Post team roster in #team-info
- ☐ Run a 15-minute kickoff meeting in Meeting Room explaining the workflow
- ☐ Have each member run /checkin to test the bot
- ☐ Assign each member one task using /task-create
- ☐ Demonstrate /report-submit at end of day
- ☐ Schedule the first weekly sync using /meeting-schedule

Phase 4: First Week Monitoring (ongoing)

- ☐ Day 1: Verify everyone checked in; address any bot issues
- ☐ Day 2: Confirm daily reports are being submitted
- ☐ Day 3: Mid-week pulse check in #task-progress
- ☐ Day 4: Review task completion rate; assign weekend tasks if any
- ☐ Day 5: Read weekly summary; run weekly sync; announce leaderboard
- ☐ Week 2: Start bi-weekly 1-on-1s; begin promoting eligible Juniors